UNIVERSITY INSTITUTE OF COMPUTING

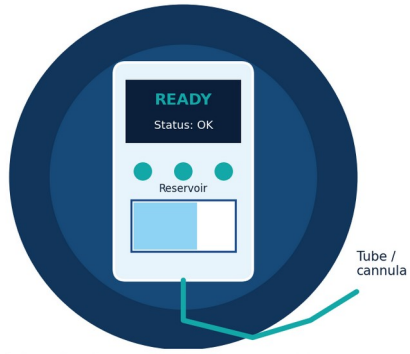
Bachelor of Computer Applications (BCA)

PROJECT BASED LEARNING (PBL) / CASE STUDY REPORT

on

INSULIN PUMP CONTROL SYSTEM

An Educational Software Engineering Case Study



Schematic of a wearable insulin pump (not to scale)

Subject: Software Engineering

Name	Ranjan Kalyan
UID	25Bcd10258
Course	BCA
Semester	3rd
Submitted To	Jitendra Kumar
Academic Session	2026–2027

University Institute of Computing

CERTIFICATE

This is to certify that the PBL / Case Study report entitled “Insulin Pump Control System” is a bonafide work carried out by Ranjan Kalyan, UID 25Bcd10258, a student of BCA, 3rd Semester, University Institute of Computing, in partial fulfilment of the requirements of the subject Software Engineering during the Academic Session 2026–2027.

The work has been completed under my guidance and, to the best of my knowledge, has not been submitted elsewhere for any other purpose. This report is an educational software engineering case study.

_____

Jitendra Kumar
Subject Teacher / Project Guide

Date: _____ Place: _____

DECLARATION

I, Ranjan Kalyan, UID 25Bcd10258, a student of BCA, 3rd Semester, University Institute of Computing, hereby declare that this PBL / Case Study report entitled “Insulin Pump Control System” is my own work, prepared as part of the subject Software Engineering for the Academic Session 2026–2027.

The information in this report has been collected from textbooks and reliable sources listed in the References section. I have not copied any report or project of another student. I further declare that this is an educational case study; it does not contain medical advice, insulin dose calculations or instructions for any real insulin pump.

Date: _____

Place: _____

Signature of Student
Ranjan Kalyan (UID: 25Bcd10258)

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my subject teacher and guide, Mr. Jitendra Kumar, for his valuable guidance, encouragement and support throughout this case study. His explanations of Software Engineering concepts helped me understand how real systems are planned, designed and tested.

I am thankful to the University Institute of Computing for giving me the opportunity to work on a Project Based Learning (PBL) topic. I also thank the Head of the Department and all faculty members for creating a good learning environment.

Finally, I thank my family and friends for their constant support and motivation during the preparation of this report.

Ranjan Kalyan
BCA, 3rd Semester

ABSTRACT

An insulin pump is a small wearable electronic device that delivers insulin under the control of embedded software. Because the software works continuously and directly affects the safety of a person, it is a classic example of a safety-critical system. This report presents the “Insulin Pump Control System” as an educational Software Engineering case study.

The report explains the problem statement, objectives and scope, compares a traditional manual approach with a proposed software-controlled design, and describes the main system components. It documents the software requirements and uses standard software engineering models: a layered architecture, a flowchart, a UML use case diagram, Data Flow Diagrams (DFD), and UML class, sequence and state diagrams. The Software Development Life Cycle (SDLC), an implementation overview, safety and risk management, sample test cases, and ethical considerations are also discussed, followed by advantages, limitations and future scope.

This work is purely educational. It does not provide medical advice, insulin dose calculations or instructions for changing the settings of any real pump. Its aim is to help a BCA student understand how requirements, design, safety and testing come together when building dependable software.

Keywords: *Insulin pump, embedded software, safety-critical system, SDLC, UML, DFD, software testing, risk management.*

TABLE OF CONTENTS

Certificate....i
Declaration....ii
Acknowledgment....iii
Abstract....iv
Table of Contents....v
List of Figures....vi
List of Tables....vii
Abbreviations and Glossary....viii
Chapter 1: Introduction....1
Chapter 2: Problem Statement....2
Chapter 3: Objectives....2
Chapter 4: Scope of the Project....3
Chapter 5: Existing System....3
Chapter 6: Proposed System....4
Chapter 7: System Components....5
Chapter 8: Software Requirements....5
Chapter 9: System Architecture....6
Chapter 10: Working of the System....7
Chapter 11: System Flowchart....8
Chapter 12: UML Use Case Diagram....9
Chapter 13: Data Flow Diagram (DFD)....10
Chapter 14: UML Design Models....10
Chapter 15: Software Development Life Cycle (SDLC)....12
Chapter 16: Implementation Overview....13
Chapter 17: Safety and Risk Management....14
Chapter 18: Software Testing....14
Chapter 19: Ethical and Professional Considerations....15
Chapter 20: Advantages....15
Chapter 21: Limitations....16
Chapter 22: Future Scope....16
Chapter 23: Conclusion....16
References....18
Appendix A: Requirements Traceability Matrix....19

LIST OF FIGURES

Figure 1.1: Simplified schematic of a wearable insulin pump....1
Figure 6.1: Conceptual block diagram of the proposed system....4
Figure 7.1: Hardware components around the processor....5
Figure 9.1: Layered software architecture of the insulin pump control system....6
Figure 11.1: Flowchart of the insulin pump control cycle....8
Figure 12.1: UML use case diagram....9
Figure 13.1: Level 0 DFD....10
Figure 13.2: Level 1 DFD....10
Figure 14.1: UML class diagram of the main classes....11
Figure 14.2: UML sequence diagram for a valid request....11
Figure 14.3: UML state diagram of the pump controller....12
Figure 15.1: SDLC phases.... 12

LIST OF TABLES

Table 2.1: Main challenges..... 2
Table 6.1: Existing versus proposed system..... 4
Table 7.1: Components and their purpose..... 5
Table 8.1: Functional requirements..... 5
Table 8.2: Non-functional requirements..... 6
Table 9.1: Software modules and responsibilities..... 7
Table 10.1: Examples of system behaviour..... 7
Table 12.1: Actors and use cases..... 9
Table 12.2: Use case description: Acknowledge Alarm..... 9
Table 15.1: SDLC phases and activities..... 12
Table 17.1: Safety measures..... 14
Table 17.2: Sample risk table..... 14
Table 18.1: Sample test cases..... 14
Table A.1: Requirement traceability..... 19

ABBREVIATIONS AND GLOSSARY

Abbreviation	Full Form
BCA	Bachelor of Computer Applications
PBL	Project Based Learning
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
DFD	Data Flow Diagram
FR / NFR	Functional / Non-Functional Requirement
UI	User Interface
PIN	Personal Identification Number
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization

Term	Meaning
Embedded system	Computer system built into a device to do a specific job.
Safety-critical system	A system whose failure could cause harm.
Actor	A user or external system that interacts with the system.
Safe state	A condition where risky activity is stopped after a fault.
Input validation	Checking that input is correct and allowed.
Data store	A place where data is kept, such as a log.

Chapter 1: Introduction

1.1 Background

Software now controls many devices that we use in daily life. In some devices, such as medical equipment, a software mistake can be dangerous. Such systems are called safety-critical systems, and they must be built with extra care. The insulin pump control system is one of the best-known examples used in Software Engineering courses.

1.2 What is Diabetes?

Diabetes is a long-term health condition in which the body cannot make or use insulin properly. Insulin is a hormone that helps the body use glucose (sugar) from food as energy. People with diabetes need regular monitoring and care, which is decided by their doctors.

1.3 What is an Insulin Pump?

An insulin pump is a small wearable electronic device that delivers insulin through a thin tube. It contains a processor, a display, buttons, a battery, an insulin reservoir, a delivery mechanism and alarms. The device is run by embedded software, which is the focus of this report.

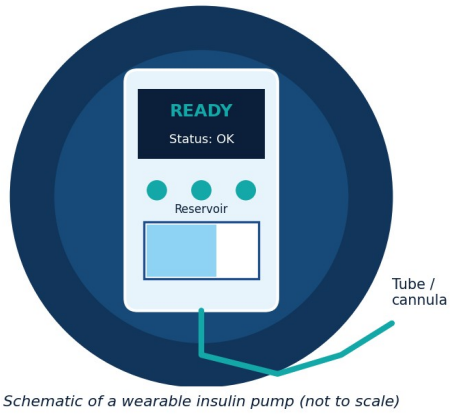


Figure 1.1: Simplified schematic of a wearable insulin pump

1.4 About this Case Study

This report studies the software side of the pump only. It applies Software Engineering concepts such as requirements, architecture, UML, DFD, SDLC and testing. It is an educational case study and not a medical treatment guide.

1.5 Organisation of the Report

Chapters 2 to 4 define the problem, objectives and scope. Chapters 5 to 8 describe the existing and proposed systems, components and requirements. Chapters 9 to 14 present the design models. Chapters 15 to 19 cover SDLC, implementation, safety, testing and ethics. Chapters 20 to 23 give advantages, limitations, future scope and the conclusion.

Chapter 2: Problem Statement

An insulin pump works continuously and automatically, without the user checking every action. If the control software fails, gives a wrong command or ignores a fault, it can harm the user. Therefore, a reliable control system is very important.

2.1 Why a Reliable Control System is Needed

- Safety-critical: a software fault can harm the user.
- Real-time: the system must respond within the required time.
- Reliability: the system must keep working without failure.
- Human error: wrong or accidental inputs must be detected.
- Fault handling: battery, motor or reservoir problems must raise alarms.

2.2 Problem Definition

The problem is to design software for an insulin pump that accepts user input, validates it, checks safety rules, controls the pump hardware, shows messages, raises alarms and records events, in a dependable way.

2.3 Key Challenges for Software Engineers

Table 2.1: Main challenges

Challenge	Meaning
Correctness	Every function must behave exactly as specified.
Timeliness	The system must react to inputs and faults on time.
Fault tolerance	The system must stay safe even when a part fails.
Usability	Simple screens reduce the chance of user mistakes.
Traceability	Every requirement should be linked to design and tests.

Chapter 3: Objectives

The main objectives of this case study are:

- To understand how an insulin pump control system works.
- To identify the functional and non-functional requirements of the system.
- To design the system using an architecture diagram, flowchart, UML diagrams and DFD.
- To apply SDLC phases and basic software testing ideas.
- To study safety and risk management in medical software.
- To understand ethical and professional responsibility in safety-critical software.

- To present the work in a clear report and presentation.

Chapter 4: Scope of the Project

Scope defines what this case study covers and what it does not cover.

4.1 Included

- Requirements analysis (functional and non-functional).
- Software architecture and module design.
- Flowchart, UML diagrams and Data Flow Diagrams.
- SDLC explanation, pseudocode and sample test cases.
- Safety, alarms, input validation and security discussion.

4.2 Excluded

- Building real hardware or electronic circuits.
- Insulin dose calculations of any kind.
- Medical advice, diagnosis or treatment guidance.
- Instructions to change real pump settings.
- Clinical trials and regulatory approval.

4.3 Assumptions

- All hardware parts are available and work as described.
- Safety limits are supplied by a qualified clinician, not by this software project.
- The case study is a design exercise, so no real patient data is used.

Chapter 5: Existing System

In the traditional approach, insulin is given manually by the user using a syringe or pen. The user performs each step personally. The simplified flow is: check the situation manually, decide the timing, prepare the syringe or pen, and inject manually.

5.1 Limitations of the Existing System

- It depends on the user’s memory and attention.
- Human error is possible in any manual step.
- There are no automatic alerts or warnings.
- There is little automatic record keeping.

5.2 Software Engineering View

From a software point of view, the existing approach has no software at all, so there is nothing to validate, monitor or log. This chapter does not compare medical outcomes; it only shows why a software-controlled design is a useful case study.

Chapter 6: Proposed System

The proposed system is a conceptual, software-controlled insulin pump. User input and sensor data pass through an input validator to the control software. The control software sends commands to the pump driver. A safety monitor watches the system, and it can trigger the display and alarm and store events in an event log.

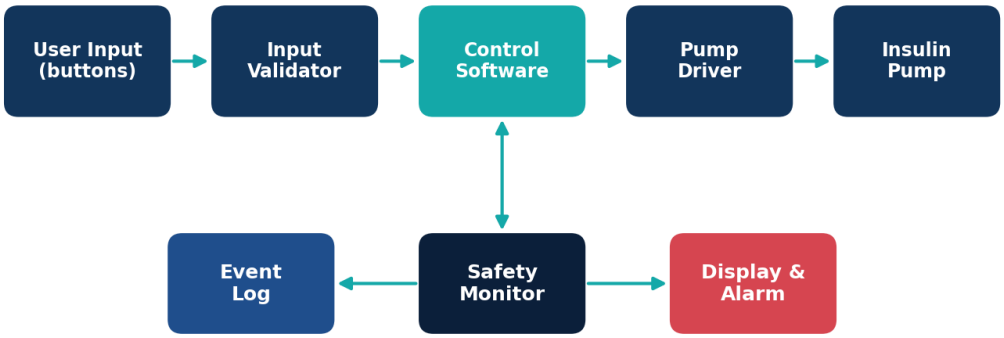


Figure 6.1: Conceptual block diagram of the proposed system

6.1 Key Features

- Software-controlled operation.
- Every request is checked before it is acted upon.
- Alerts are raised when a fault is found.
- Events are stored in a log for later review.

Table 6.1: Existing versus proposed system

Point	Existing System	Proposed System
Control	Manual, by the user	Software-controlled
Checks	None	Input and safety checks
Alerts	No automatic alerts	Alarms for faults
Records	Little or none	Event log

Chapter 7: System Components

The pump has several hardware components. Our software controls and monitors all of them.

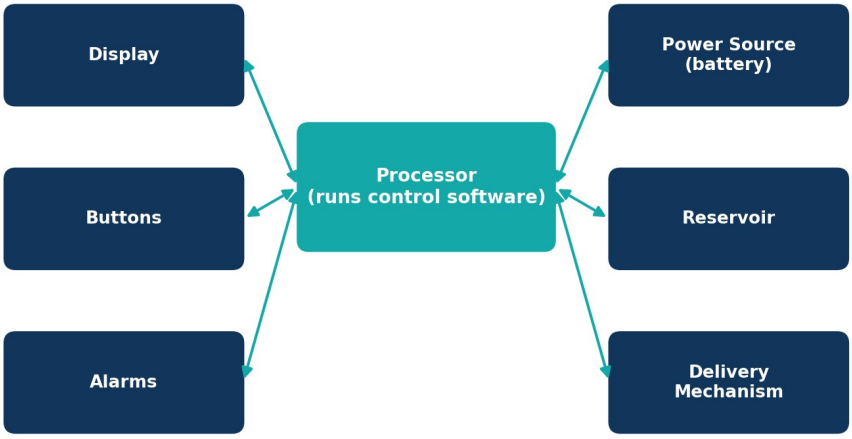


Figure 7.1: Hardware components around the processor

Table 7.1: Components and their purpose

Component	Purpose
Processor (microcontroller)	Runs the control software.
Display	Shows status and messages to the user.
Buttons	Take input from the user.
Power source (battery)	Supplies power; its level is monitored by software.
Insulin reservoir	Holds the insulin supply.
Delivery mechanism	Motor and tube that deliver insulin from the reservoir.
Alarms	Sound or vibration to warn the user about faults.

Chapter 8: Software Requirements

Requirements describe what the system must do (functional) and how well it must do it (non-functional).

Table 8.1: Functional requirements

ID	Requirement
FR1	The system shall accept user input through buttons.
FR2	The system shall validate every input before using it.
FR3	The system shall control the pump motor through the pump driver.
FR4	The system shall display status and messages.
FR5	The system shall raise an alarm when a fault is detected.
FR6	The system shall record events in a log.

Table 8.2: Non-functional requirements

Quality	Description
Reliability	The software should work continuously without failure.
Safety	On a fault, the system should move to a safe state.
Real-time response	Inputs and faults should be handled on time.
Usability	Screens and buttons should be simple to use.
Security	Settings and data should be protected from misuse.
Maintainability	The code should be easy to update and fix.

8.3 Constraints

- The software must run on a small processor with limited memory and power.
- The device must keep working on battery power.
- Safety rules must always have priority over convenience.

Chapter 9: System Architecture

A layered architecture is used. Each layer talks only to the layer next to it, which keeps the design simple and easier to test.

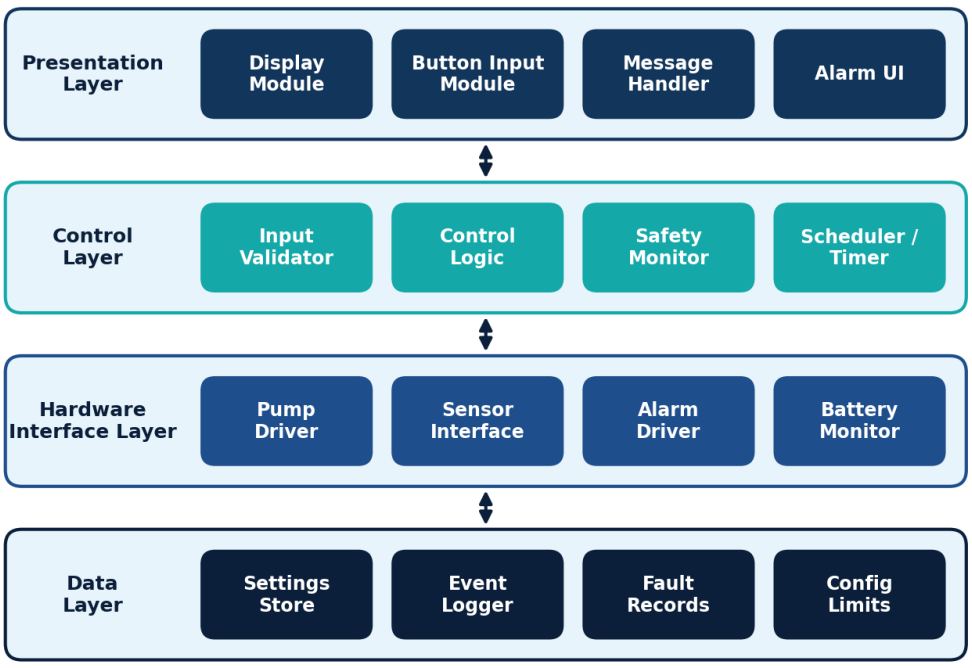


Figure 9.1: Layered software architecture of the insulin pump control system

Table 9.1: Software modules and responsibilities

Module	Responsibility
Display Module / Message Handler	Show status, warnings and messages.
Button Input Module	Read button presses from the user.
Input Validator	Check that inputs are correct and allowed.
Control Logic	Decide the next action of the pump.
Safety Monitor	Watch for faults and enforce safety rules.
Scheduler / Timer	Run tasks at the right time.
Pump Driver	Send commands to the pump hardware.
Sensor Interface / Battery Monitor	Read sensor and battery information.
Alarm Driver	Control the buzzer or vibration alert.
Settings Store / Event Logger	Store limits, settings and event records.

Chapter 10: Working of the System

The workflow of the system is explained step by step below.

1. The device is powered on and runs a self-test.
2. The software reads user input and sensor data.
3. The input is validated for correctness.
4. The request is checked against stored safety limits.
5. If everything is correct, a command is sent to the pump driver.
6. The display is updated, the event is logged, and the cycle repeats.

If a fault is found at any step, an alarm is raised and the system moves to a safe state. The cycle continues as long as the device is powered on.

10.1 Normal and Fault Situations

Table 10.1: Examples of system behaviour

Situation	System Response
Normal request	Validate → check limits → command pump → log → update display.
Invalid input	Reject input, show an error message, log the event.
Fault detected	Raise alarm, stop activity and enter the safe state.
Low battery	Show a warning and raise an alert.

Chapter 11: System Flowchart

The flowchart shows the main control cycle. The oval is the start, rectangles are process steps, diamonds are decisions, and the red boxes are the alarm path.

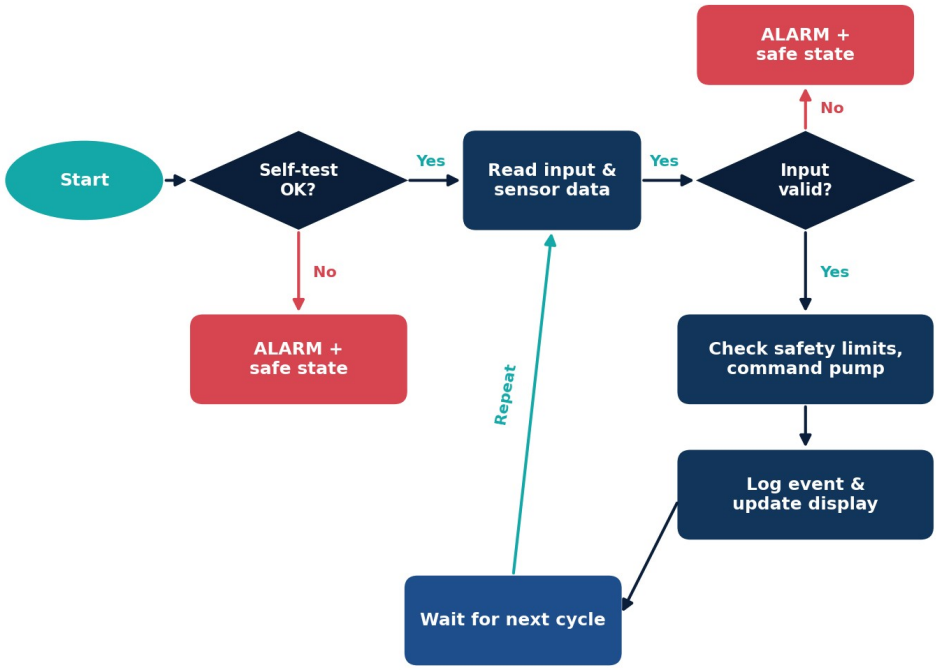


Figure 11.1: Flowchart of the insulin pump control cycle

Explanation: After start, a self-test is done. If it fails, an alarm is raised and the system goes to a safe state. If it passes, inputs are read and validated. Valid requests are checked against limits, the pump is commanded, the event is logged, and the system waits for the next cycle.

Chapter 12: UML Use Case Diagram

A use case diagram shows the actors (users or external systems) and how they interact with the system.

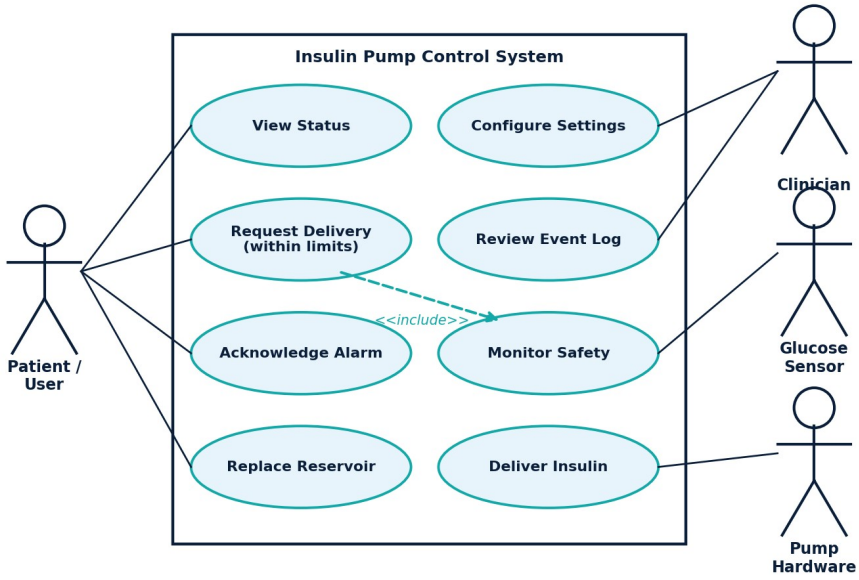


Figure 12.1: UML use case diagram

Table 12.1: Actors and use cases

Actor	Use Cases
Patient / User	View status, request delivery (within limits), acknowledge alarm, replace reservoir.
Clinician	Configure settings, review event log.
Glucose Sensor	Provides readings used by Monitor Safety.
Pump Hardware	Receives commands to deliver insulin.

The dashed line shows that “Request Delivery” includes the “Monitor Safety” check.

Table 12.2: Use case description: Acknowledge Alarm

Item	Description
Actor	Patient / User
Precondition	An alarm is active.
Main flow	1. Alarm sounds. 2. User reads the message. 3. User presses the acknowledge button. 4. System stops the sound and logs the action.
Alternate flow	If the fault is still present, the system keeps the safe state and shows the message again.
Postcondition	Alarm is acknowledged and recorded in the log.

Chapter 13: Data Flow Diagram (DFD)

A DFD shows how information moves through a system. Rectangles are external entities, circles are processes, open boxes are data stores, and arrows are data flows.

13.1 Level 0 (Context Diagram)

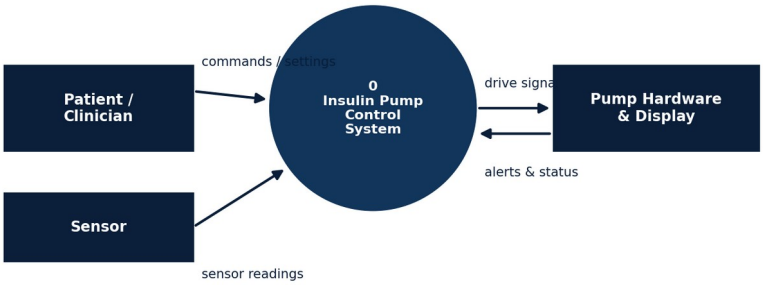


Figure 13.1: Level 0 DFD

13.2 Level 1 DFD

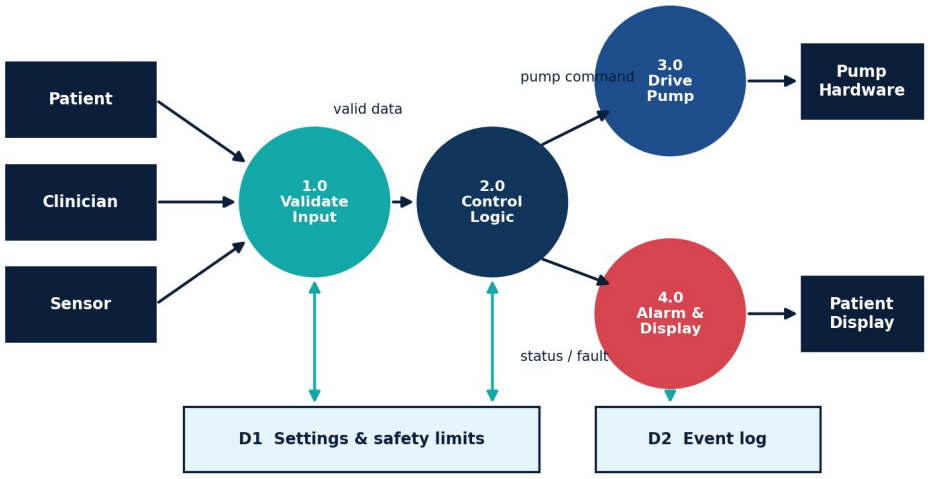


Figure 13.2: Level 1 DFD

Explanation: Commands, settings and sensor readings enter process 1.0 (Validate Input). Valid data goes to 2.0 (Control Logic), which sends a pump command to 3.0 (Drive Pump) and status or fault information to 4.0 (Alarm & Display). D1 stores settings and safety limits; D2 stores the event log.

Chapter 14: UML Design Models

Besides the use case diagram, three more UML diagrams explain the design: the class diagram (structure), the sequence diagram (order of messages) and the state diagram (behaviour over time).

14.1 Class Diagram

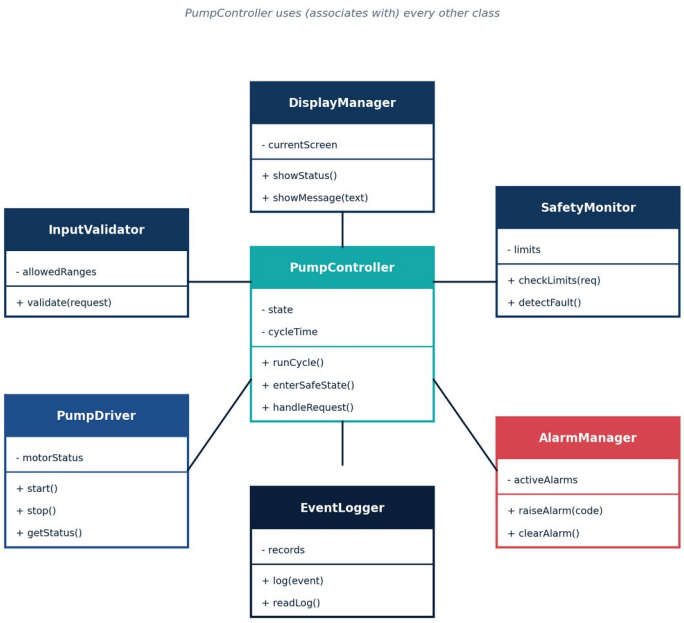


Figure 14.1: UML class diagram of the main classes

14.2 Sequence Diagram

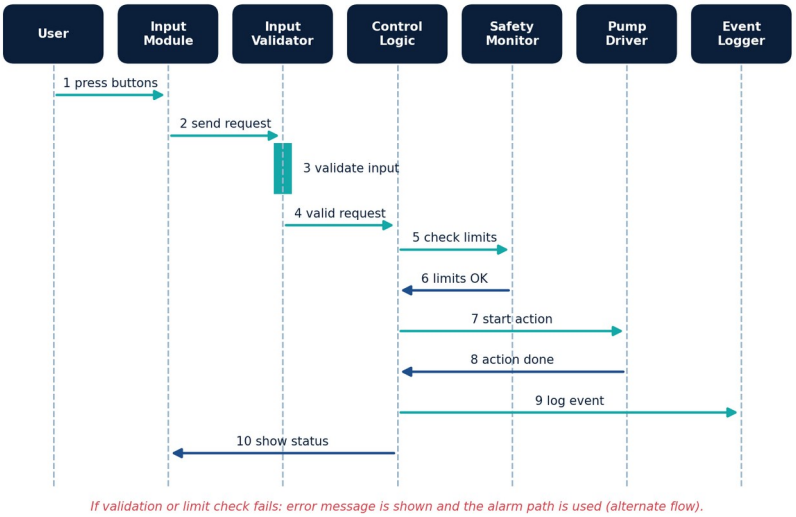


Figure 14.2: UML sequence diagram for a valid request

14.3 State Diagram

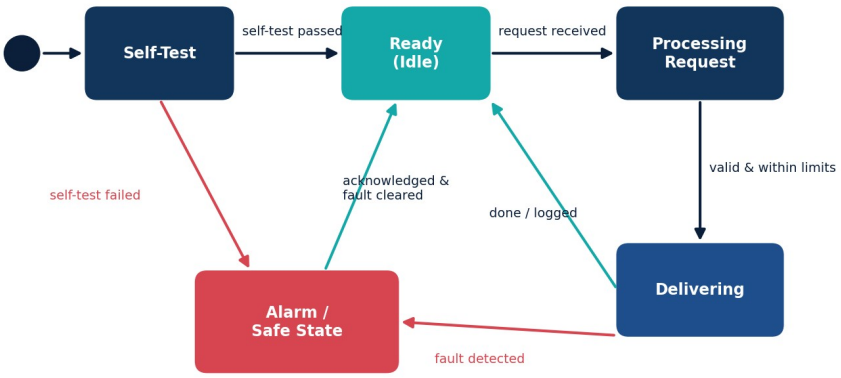


Figure 14.3: UML state diagram of the pump controller

Explanation: The controller starts in Self-Test. If it passes, it moves to Ready. A valid request moves it to Processing and then Delivering; after the action is done and logged it returns to Ready. Any fault sends it to the Alarm / Safe State, from which it returns to Ready only after the alarm is acknowledged and the fault is cleared.

Chapter 15: Software Development Life Cycle (SDLC)

SDLC is the step-by-step process of building software. For safety-critical software, careful planning and documentation are very important.



Figure 15.1: SDLC phases

Table 15.1: SDLC phases and activities

Phase	Activity for this case study
Planning	Define goals, scope and feasibility.
Requirements	Collect and document functional and non-functional needs.
Design	Prepare architecture, modules, flowchart, UML and DFD.
Implementation	Write code module by module.
Testing	Unit, integration, system and acceptance testing.

Phase	Activity for this case study
Deployment	Release the software to the target device.
Maintenance	Fix defects and make updates.

A plan-driven model with strong documentation (such as the Waterfall or V-model) suits safety-critical work because each phase is reviewed before the next begins. Feedback can still move work back to an earlier phase when a problem is found.

Chapter 16: Implementation Overview

This chapter gives a simple idea of how the design can be coded. It is a teaching sketch only. It uses no real medical values.

16.1 Suggested Technology

- Embedded C or C++ is commonly used for small, real-time devices.
- A Python simulation can be used in a classroom to test the logic without hardware.
- Each module in the architecture becomes a separate source file or class.

16.2 Main Control Cycle (Pseudocode)

```
LOOP every cycle:
  IF self_test_failed OR fault_detected:
      ENTER_SAFE_STATE()
      RAISE_ALARM(fault_code)
      LOG(fault_code)
      CONTINUE

  request = READ_INPUT()
  IF NOT VALIDATE(request):
      SHOW_MESSAGE("Invalid input")
      LOG("invalid input")
      CONTINUE

  IF NOT WITHIN_SAFETY_LIMITS(request):
      RAISE_ALARM(limit_code)
      LOG(limit_code)
      CONTINUE

  SEND_COMMAND(pump_driver, request)
  LOG("action done")
  UPDATE_DISPLAY()
  WAIT_FOR_NEXT_CYCLE()
```

Note: WITHIN_SAFETY_LIMITS uses limits stored in the settings store. How those limits are chosen is a medical decision and is outside this project.

16.3 Coding Practices

- Keep functions small and test each one alone.
- Check every input before using it.

- Never ignore return codes or error states.
- Use comments and meaningful names for easy maintenance.

Chapter 17: Safety and Risk Management

Safety is the most important quality of this system. Risk management means identifying, assessing, reducing and monitoring risks.

Table 17.1: Safety measures

Area	Measures
Fault detection	Self-test at start-up; watch battery, motor and reservoir status.
Alarms	Clear sound or visual warning; move to a safe state.
Input validation	Reject out-of-range values; ask for confirmation of important actions.
Security	PIN or lock for settings; secure any wireless link; keep a log of changes.

Table 17.2: Sample risk table

Risk	Possible Effect	Control
Invalid user input	Wrong command accepted	Input validation and confirmation
Low battery	Device stops working	Battery monitor and early warning alarm
Motor or driver fault	Delivery fails	Fault detection, alarm, safe state
Unauthorised change	Settings misused	PIN lock and change log
Software defect	Unexpected behaviour	Reviews and thorough testing

Real medical device software must follow standards such as IEC 62304 (software life cycle) and ISO 14971 (risk management). This case study does not claim compliance with them.

Chapter 18: Software Testing

Testing checks that the software does what it should. The test cases below are samples only and use no real medical values.

Table 18.1: Sample test cases

ID	Test Case	Input / Condition	Expected Result
TC01	Start-up self-test	Device switched on	Self-test passes; “Ready” is shown
TC02	Input validation	Value outside allowed range	Input rejected; error message shown
TC03	Wrong button sequence	Keys pressed in wrong order	Ignored; user asked to retry

ID	Test Case	Input / Condition	Expected Result
TC04	Low battery	Battery below threshold	Warning alarm raised
TC05	Low reservoir	Reservoir nearly empty	Alert shown to the user
TC06	Motor fault	Pump driver reports a fault	Alarm raised; delivery stops safely
TC07	Event logging	Any action performed	Event saved with time stamp

18.1 Levels of Testing

- Unit testing: each module is tested alone.
- Integration testing: modules are tested together.
- System testing: the complete system is tested.
- Acceptance testing: checks that requirements are met.

18.2 Testing Methods

- Black-box testing: test inputs and outputs without looking at the code.
- White-box testing: test internal paths and decisions in the code.
- Boundary value testing: test values at and near the allowed limits.

Chapter 19: Ethical and Professional Considerations

Software engineers who build medical software have a special responsibility. The ACM/IEEE Software Engineering Code of Ethics asks engineers to act in the public interest.

- Safety first: never release software that is not properly tested.
- Honesty: report known faults and limitations clearly.
- Privacy: protect any personal health data.
- Competence: work only within one’s skills and ask experts, such as doctors, for medical decisions.
- Accountability: keep proper documents so that decisions can be reviewed.

This project follows these ideas by staying within software design and avoiding any medical advice.

Chapter 20: Advantages

A software-controlled insulin pump design offers the following advantages from a software engineering point of view:

- Fewer manual steps for the user.
- Consistent, software-controlled operation.

- Automatic alerts on faults.
- Better record keeping through the event log.
- Less dependence on the user’s memory and attention.
- Faster detection of faults, since the software checks the system every cycle.
- Input validation catches wrong or out-of-range entries before they are used.
- Modular design makes testing, debugging and updates easier.
- Tested modules can be reused in other embedded projects.
- Models such as UML and DFD make the system easy to understand and explain.
- Clear documentation helps team members work together.
- A layered architecture makes it easy to add new features later.
- The case study helps students learn the real steps of the SDLC.

Chapter 21: Limitations

This case study and the proposed design also have some limitations:

- The system depends on correct software; one hidden defect can cause problems.
- Hardware is not built in this case study.
- It needs strict testing and approval before real use.
- Battery or device faults are still possible.
- Limits and medical settings are outside the scope of this project.
- Only a conceptual design and sample test cases are presented.
- Testing cannot prove that every possible fault has been found.
- Development and testing of safety-critical software needs more time, effort and cost.
- Skilled engineers and medical experts are needed to build a real system.
- A small processor and limited battery restrict how complex the software can be.
- Software updates must be controlled carefully, because a new change can introduce new errors.
- Users need training to use the device, its alarms and its messages correctly.
- Any wireless connection could create privacy and security risks if not protected.
- Results from a classroom simulation may differ from real-world conditions.

Chapter 22: Future Scope

- Mobile app connection with strong security.
- Better and more accessible user interface.
- Smarter fault detection.

- Secure reports for review by clinicians.
- A working simulation in Python or C++ for classroom testing.
- Formal verification and fuller testing of the safety rules.

Chapter 23: Conclusion

This case study showed how Software Engineering concepts apply to a safety-critical system. Requirements, a layered architecture, a flowchart, UML and DFD models, the SDLC, risk management and test cases together help to build dependable software. Alarms, input validation and fault checks are essential to reduce risk.

The study also showed why careful testing, documentation and ethics matter when software can affect human safety. This is an educational project; it is not a medical treatment guide, and it does not provide dose calculations or instructions for any real pump.

The work has improved my understanding of the Software Engineering subject and how its models are used in real problems.

References

[1] I. Sommerville, Software Engineering, Pearson Education (insulin pump case study).

[2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, McGraw-Hill.

[3] P. Jalote, An Integrated Approach to Software Engineering, Springer.

[4] Object Management Group (OMG), Unified Modeling Language (UML) Specification, omg.org.

[5] IEC 62304: Medical device software – Software life cycle processes.

[6] ISO 14971: Medical devices – Application of risk management to medical devices.

[7] ACM/IEEE-CS, Software Engineering Code of Ethics and Professional Practice, acm.org.

[8] World Health Organization (WHO), Diabetes fact sheet, who.int.

[9] National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK), insulin pump information, niddk.nih.gov.

Appendix A: Requirements Traceability Matrix

Traceability links each requirement to a module and a test case, so nothing is left untested.

Table A.1: Requirement traceability

Requirement	Module(s)	Test Case(s)
FR1 Accept user input	Button Input Module	TC03
FR2 Validate input	Input Validator	TC02, TC03
FR3 Control pump motor	Control Logic, Pump Driver	TC06
FR4 Display status	Display Module, Message Handler	TC01, TC05
FR5 Raise alarms	Safety Monitor, Alarm Driver	TC04, TC05, TC06
FR6 Log events	Event Logger	TC07